

数字电路设计 — DDCA 第 1 章数字设计基础

详细学习笔记

Digital Design and Computer Architecture: ARM® Edition

Sarah L. Harris & David Money Harris

2026 年 6 月 15 日

摘要：本文覆盖 DDCA 第 1 章全部 120 页内容，以中英双语术语形式整理考试重点，包括：数字抽象与管理复杂性、数制系统与转换、二进制补码运算、逻辑门与布尔代数、逻辑电平与噪声容限、CMOS 晶体管原理、摩尔定律及功耗分析。

课程概览与背景 [p.1–4]

课程目标 [p.4]

- 理解计算机底层 (*Understand what's under the hood*): 从晶体管到微处理器的完整数字设计链路
- 学习数字设计原理 (*Learn principles of digital design*)
- 系统化调试 (*Learn to systematically debug*) 日益复杂的设计
- 设计并构建微处理器 (*Design and build a microprocessor*)

行业背景 [p.3]

- 微处理器已彻底改变世界：手机、互联网、医疗等
- 半导体行业从 1985 年 \$210 亿增长到 2013 年 \$3060 亿

管理复杂性的艺术 [p.5–12]

抽象 (*Abstraction*) [p.5–6]

- 定义：隐藏不重要的细节，抓住主要矛盾 [p.5–6]
- 抽象层次（自顶向下）：程序 → 设备驱动 → 指令 → 寄存器 → 数据通路 → 控制器 → 加法器/存储器 → 与门/非门 → 放大器/晶体管 → 二极管/电子 [p.6]

- **考点：**抽象是数字设计的核心方法——在每一层只需要关注该层的接口和功能，忽略下层实现细节

约束 (*Discipline*) [p.7]

- **定义：**有意地限制设计选择 [p.7]
- **数字约束 (*Digital Discipline*):** 使用离散电压而非连续电压，比模拟电路更简单，可以构建更复杂的系统 [p.7]
- **数字系统替代模拟前身：**数码相机、数字电视、手机、CD [p.7]

三原则 (*The Three-Y's*) [p.8–12]

1. **层次化 (*Hierarchy*):** 系统分为模块和子模块 [p.9]
2. **模块化 (*Modularity*):** 每个模块有明确定义的功能和接口 [p.9]
3. **规整化 (*Regularity*):** 鼓励一致性，模块可方便地复用 [p.9]

实例：燧发步枪 (*Flintlock Rifle*) [p.10–12]

- **层次化：**三大模块——枪机 (lock)、枪托 (stock)、枪管 (barrel)；枪机的子模块——扳机 (trigger)、击锤 (hammer)、燧石 (flint)、打火板 (frizzen) [p.10]
- **模块化：**枪托的功能 (安装枪管和枪机) 和接口 (安装销的长度和位置) 明确定义 [p.12]
- **规整化：**可互换零件 (interchangeable parts) [p.12]

数字抽象与二进制基础 [p.13–17]

数字抽象 (*Digital Abstraction*) [p.13]

- 大多数物理变量是连续的（电压、频率、位置等） [p.13]
- **数字抽象**只考虑离散子集的值 [p.13]

分析机 (*The Analytical Engine*) [p.14–15]

- Charles Babbage 于 1834–1871 年设计，第一台数字计算机 [p.14]
- 使用机械齿轮，每个齿轮代表离散值 (0–9) [p.14]
- **Ada Lovelace:** 第一个程序员，编写了求解伯努利数的算法 [p.15]

二进制值 (*Binary Values*) [p.16]

- 两个离散值：1 (TRUE, HIGH) 和 0 (FALSE, LOW) [p.16]
- 位 (*Bit*): Binary digit (二进制数字) [p.16]
- 数字电路使用电压表示 1 和 0 [p.16]

乔治·布尔 (*George Boole, 1815–1864*) [p.17]

- 自学数学，1854 年发表《思维规律的研究》(*An Investigation of the Laws of Thought*) [p.17]
- 引入二进制变量，定义了三种基本逻辑运算：AND、OR、NOT [p.17]

数制系统 [p.18–31]

十进制与二进制 [p.18–19]

- 十进制 (Decimal): 基数为 10

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

[p.19]

- 二进制 (Binary): 基数为 2

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$$

[p.19]

2 的幂次 (*Powers of Two*) [p.20–21]

- 建议熟记 2^0 到 2^9 : 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 [p.21]
- 更高幂次: $2^{10} = 1024$, $2^{11} = 2048$, $2^{12} = 4096$, $2^{13} = 8192$, $2^{14} = 16384$, $2^{15} = 32768$ [p.21]

数制转换 (*Number Conversion*) [p.22–29]

二进制转十进制 [p.22–23]

- 加权求和法: 每位乘以对应权重
- 例: $10011_2 = 16 + 2 + 1 = 19_{10}$ [p.23]

十进制转二进制 [p.24–29]

方法 1: 最大幂减法 (Find largest power of 2) [p.25–26]

- 找出不超过该数的最大 2 的幂, 减去后重复
- 例: $53_{10} \rightarrow 32 \times 1, 21 \rightarrow 16 \times 1, 5 \rightarrow 4 \times 1, 1 \rightarrow 1 \times 1 = 110101_2$ [p.26]

方法 2: 重复除 2 取余法 (Repeatedly divide by 2) [p.26, 29]

- 不断除以 2, 余数从 LSB 到 MSB 排列
- 例 (53_{10}): $53/2=26$ R1, $26/2=13$ R0, $13/2=6$ R1, $6/2=3$ R0, $3/2=1$ R1, $1/2=0$ R1 $\Rightarrow 110101_2$ [p.26]
- 例 (75_{10}): $75 = 64 + 8 + 2 + 1 = 1001011_2$, 或除 2 法: $75/2=37$ R1, $37/2=18$ R1, $18/2=9$ R0, $9/2=4$ R1, $4/2=2$ R0, $2/2=1$ R0, $1/2=0$ R1 $\Rightarrow 1001011_2$ [p.28–29]

二进制值的范围 (Binary Values and Range) [p.30–31]

- N 位十进制数: 10^N 个值, 范围 $[0, 10^N - 1]$ [p.31]
- N 位二进制数: 2^N 个值, 范围 $[0, 2^N - 1]$ [p.31]
- 例: 3 位二进制数: $2^3 = 8$ 个值, 范围 $[0, 7] = [000_2, 111_2]$ [p.31]

十六进制数 [p.32–36]

十六进制基础 [p.32–34]

- 基数为 16, 数字 0–9, A(10), B(11), C(12), D(13), E(14), F(15) [p.32–33]
- 每位十六进制数对应 4 位二进制 (即一个 nibble) [p.33]
- 十六进制是二进制的简写形式 (Shorthand) [p.34]

十六进制转换 [p.35–36]

- 十六进制 $\rightarrow$ 二进制: 每位直接展开为 4 位二进制例: $4AF_{16} \rightarrow 0100\ 1010\ 1111_2$ [p.36]
- 十六进制 $\rightarrow$ 十进制: 按权展开

$$4AF_{16} = 16^2 \times 4 + 16^1 \times 10 + 16^0 \times 15 = 1199_{10}$$

[p.36]

- 注意: $0x4AF$ 前缀 $0x$ 表示十六进制 [p.35]

位、字节与大幂次 [p.37–40]

位与字节 [p.37]

- 位 (*Bit*): 单个二进制数字, 最左为 MSB (Most Significant Bit), 最右为 LSB (Least Significant Bit) [p.37]
- 字节 (*Byte*): 8 位, 如 10010110 [p.37]
- 半字节 (*Nibble*): 4 位, 即一个十六进制位的二进制表示 [p.37]

大幂次单位 [p.38]

- $2^{10} \approx 10^3$: 1 kilo (千) ≈ 1024
- $2^{20} \approx 10^6$: 1 mega (兆) $\approx 1,048,576$
- $2^{30} \approx 10^9$: 1 giga (吉) $\approx 1,073,741,824$
- 2^{40} : 1 tera (太/TB) = 1024 GB
- 2^{50} : 1 peta (拍/PB) = 1024 TB
- 2^{60} : 1 exa (艾/EB) = 1024 PB
- 2^{70} : 1 zetta (泽/ZB)
- 2^{80} : 1 yotta (尧/YB)

估算 2 的幂 [p.39–40]

- 技巧: 分解为已知值乘积
- $2^{24} = 2^4 \times 2^{20} \approx 16 \times 10^6 = 16 \text{ million}$ [p.40]
- 32 位变量可表示的值: $2^{32} = 2^2 \times 2^{30} \approx 4 \times 10^9 = 4 \text{ billion}$ [p.40]

二进制加法与溢出 [p.41–46]

二进制加法规则 [p.41–44]

- 逐位相加, 进位 (carry) 传递 [p.41–42]

- 例: $1011_2 + 0011_2 = 1110_2$ [p.42]
- 例: $1001_2 + 0101_2 = 1110_2$ (无溢出) [p.43-44]

溢出 (*Overflow*) [p.45–46]

- 定义: 结果太大, 超出可用位数 [p.46]
- 例: $1011_2 + 0110_2 = 10001_2$ (5 位结果溢出了 4 位宽度) [p.45]
- 数字系统使用固定位宽 (fixed number of bits), 溢出是重要的设计考量 [p.46]

有符号二进制数 [p.47–68]

原码 (*Sign/Magnitude*) [p.47–51]

- 1 位符号位 (最高位) + N-1 位数值位 [p.48-49]
- 符号位: 0 = 正, 1 = 负 [p.48]
- 例 (4 位): $+6 = 0110$, $-6 = 1110$ [p.49]
- 范围: $[-(2^{N-1} - 1), 2^{N-1} - 1]$ [p.49-50]
- 问题 1 (加法不工作): $-6 + 6$: $1110 + 0110 = 10100$ (错误) [p.51]
- 问题 2 (双零表示): $+0 = 0000$, $-0 = 1000$ (存在 $+0$ 和 -0 两个表示) [p.51]

补码 (*Two's Complement*) [p.52–60]

定义与性质 [p.52–54]

- MSB 具有权重 -2^{N-1} [p.53-54]
- 解决了原码的两大问题: 加法直接可用, 0 只有一个表示 [p.52]
- 定义式: $[x]_{\text{补}} = 2^N + x \pmod{2^N}$ [p.52]
- 例 (4 位): $x = -3 \Rightarrow [x]_{\text{补}} = 2^4 + (-3) = 13 = 1101_2$ [p.52]
- 范围: $[-2^{N-1}, 2^{N-1} - 1]$ (比原码多表示一个负数) [p.54]
- 4 位补码: 最大正数 $0111 = 7$, 最大负数 $1000 = -8$ [p.54]
- 最高位仍指示符号 (1 = 负, 0 = 非负) [p.54]

取补码操作 (*Taking the Two's Complement*) [p.55–58]

- 翻转符号的方法： [p.55–56]
 1. 按位取反 (Invert the bits)
 2. 加 1 (Add 1)
- 例：翻转 $3_{10} = 0011_2$ 的符号：取反 $\rightarrow 1100$ ，加 1 $\rightarrow 1101 = -3_{10}$ [p.56]
- 例：翻转 $6_{10} = 0110_2$ 的符号：取反 $\rightarrow 1001$ ，加 1 $\rightarrow 1010 = -6_{10}$ [p.58]
- 求补码数的十进制值：对 1001_2 再取补码 $\rightarrow 0111 = 7$ ，故 $1001_2 = -7_{10}$ [p.57–58]

补码加法 [p.59–60]

- 符号位参与运算，溢出位丢弃 [p.59]
- 例： $6 + (-6)$: $0110 + 1010 = 1\ 0000$ ，忽略进位得 0000 （正确：0） [p.60]
- 例： $(-2) + 3$: $1110 + 0011 = 1\ 0001$ ，忽略进位得 0001 （正确：1） [p.60]

位宽扩展 (*Increasing Bit Width*) [p.61–63]**符号扩展 (*Sign-Extension*) [p.62]**

- 将符号位复制到新增的 MSB 位，数值保持不变 [p.62]
- 例（4 位 $\rightarrow$ 8 位）： $3 = 0011 \rightarrow 00000011$ [p.62]
- 例（4 位 $\rightarrow$ 8 位）： $-5 = 1011 \rightarrow 11111011$ [p.62]

零扩展 (*Zero-Extension*) [p.63]

- 新增 MSB 位补 0，仅适用于无符号数 [p.63]
- 注意：对有符号负数使用零扩展会改变值！例： $1011 = -5_{10}$ ，零扩展后 $00001011 = 11_{10} \neq -5_{10}$ [p.63]

数制系统比较 [p.64–68]

数制系统	范围 (N 位)
无符号数 (<i>Unsigned</i>)	$[0, 2^N - 1]$
原码 (<i>Sign/Magnitude</i>)	$[-(2^{N-1} - 1), 2^{N-1} - 1]$
补码 (<i>Two's Complement</i>)	$[-2^{N-1}, 2^{N-1} - 1]$

4 位表示对照 [p.64–65]:

- 无符号数: 0000(0) $\rightarrow$ 1111(15), 共 16 个值
- 补码: 1000(-8) $\rightarrow$ 0111(7), 共 16 个值, 零唯一 (0000)
- 原码: 1111(-7) $\rightarrow$ 0111(7), 0000(+0) 和 1000(-0) 双零, 共 15 个不同值

移码 (*Biased/Offset Code*) [p.65–66]

- 定义: $[x]_{\text{移}} = 2^{N-1} + x$ [p.66]
- 常用于浮点数阶码表示 [p.66]
- 实际上就是补码的符号位取反 [p.66]
- 例 (4 位, $x = -3$): $[x]_{\text{移}} = 2^3 + (-3) = 0101_2$ [p.66]
- 优点: 比较补码对应数值大小时更直观方便 [p.65]

反码 (*One's Complement*) [p.66]

- 负数: $[x]_{\text{反}} = 2^N - 1 + x$ (原码数值位取反) [p.66]
- 主要用于原码求补码或补码求原码的过渡码 [p.66]
- 例 (4 位, $x = -3$): $[x]_{\text{反}} = 2^4 - 1 + (-3) = 1100_2$ [p.66]

逻辑门 [p.69–79]

概述 [p.69]

- 实现逻辑函数: NOT (非)、AND (与)、OR (或)、NAND (与非)、NOR (或非) 等 [p.69]
- 单输入门: NOT 门、BUF (缓冲器) [p.69]
- 双输入门: AND、OR、XOR、NAND、NOR、XNOR [p.69]
- 多输入门: 三输入及以上 [p.69]

单输入门 [p.70–71]

NOT 门（非门/反相器） [p.70–71]

- 布尔表达式: $Y = \overline{A}$
- 真值表: $A = 0 \Rightarrow Y = 1, A = 1 \Rightarrow Y = 0$ [p.71]

BUF 门（缓冲器） [p.70–71]

- 布尔表达式: $Y = A$
- 真值表: $A = 0 \Rightarrow Y = 0, A = 1 \Rightarrow Y = 1$ [p.71]
- 用途: 增强驱动能力

双输入基本门 [p.72–75]

AND 门（与门） [p.72–73]

- $Y = AB$ （逻辑乘）
- 只有全部输入为 1 时输出才为 1
- 真值表: $00 \rightarrow 0, 01 \rightarrow 0, 10 \rightarrow 0, 11 \rightarrow 1$ [p.73]

OR 门（或门） [p.72–73]

- $Y = A + B$ （逻辑加）
- 任一输入为 1 时输出即为 1
- 真值表: $00 \rightarrow 0, 01 \rightarrow 1, 10 \rightarrow 1, 11 \rightarrow 1$ [p.73]

XOR 门（异或门） [p.74–75]

- $Y = A \oplus B$ （模 2 加）
- 输入不同时输出为 1
- 真值表: $00 \rightarrow 0, 01 \rightarrow 1, 10 \rightarrow 1, 11 \rightarrow 0$ [p.75]
- 考点: 多输入 XOR = 奇校验 (Odd parity): 输入中 1 的个数为奇数时输出为 1 [p.77]

NAND 门（与非门） [p.74–75]

- $Y = \overline{AB}$
- AND 的反相，全 1 出 0
- 真值表：00→1, 01→1, 10→1, 11→0 [p.75]

NOR 门（或非门） [p.74–75]

- $Y = \overline{A + B}$
- OR 的反相，有 1 出 0
- 真值表：00→1, 01→0, 10→0, 11→0 [p.75]

XNOR 门（同或门/异或非） [p.74–75]

- $Y = \overline{A \oplus B}$
- 输入相同时输出为 1
- 真值表：00→1, 01→0, 10→0, 11→1 [p.75]

多输入门 [p.76–77]

- NOR3: $Y = \overline{A + B + C}$ ，全 0 出 1 [p.76–77]
- AND3: $Y = ABC$ ，全 1 出 1 [p.76–77]
- 考点：多输入 XOR 的行为 = 奇校验，即输出 = 输入中 1 个数的奇偶性 [p.77]

逻辑门国标与仿真 [p.78–79]

- 中国国家标准有对应的逻辑门符号体系 [p.78]
- 推荐使用 Digital 仿真工具学习构建数字电路 [p.79]

逻辑电平与噪声容限 [p.80–89]

逻辑电平 (*Logic Levels*) [p.80–81]

- 离散电压表示 1 和 0 [p.80]
- 例：0 = GND (0V)，1 = V_{DD} (如 5V) [p.80]
- 用电压范围 (Range) 而非单一值来表示 0 和 1 [p.81]
- 输入和输出使用不同的范围，以容纳噪声 [p.81]

噪声 (*Noise*) [p.82]

- 定义：任何降低信号质量的因素 [p.82]
- 来源：电阻、电源噪声、邻近导线耦合 (coupling) 等 [p.82]
- 例：门输出 5V，但因长导线电阻，接收端只收到 4.5V [p.82]

静态约束 (*The Static Discipline*) [p.83]

- 核心原则：在逻辑有效 (logically valid) 的输入下，每个电路元件必须产生逻辑有效的输出 [p.83]
- 使用有限的电压范围 (limited ranges) 表示离散值 [p.83]

噪声容限 (*Noise Margins*) [p.84–85]

- V_{OH} ：输出高电平最小值 (Output High)
- V_{OL} ：输出低电平最大值 (Output Low)
- V_{IH} ：输入高电平最小值 (Input High)
- V_{IL} ：输入低电平最大值 (Input Low)
- 禁止区 (*Forbidden Zone*)： V_{IL} 与 V_{IH} 之间的电压范围 [p.84]

噪声容限公式 [p.85]：

- 高噪声容限： $NM_H = V_{OH} - V_{IH}$
- 低噪声容限： $NM_L = V_{IL} - V_{OL}$
- 噪声容限越大，允许的噪声越大，抗干扰能力越强 [p.85]
- 低噪声容限必须大于 0，防止低电平进入禁止区 [p.85]
- V_{IL} 必须大于 V_{OL} [p.84]

DC 传输特性 (*DC Transfer Characteristics*) [p.86–87]

- 理想缓冲器： $NM_H = NM_L = V_{DD}/2$ ， $V_{IL} = V_{IH} = V_{DD}/2$ [p.86]
- 实际缓冲器： $NM_H, NM_L < V_{DD}/2$ ，存在过渡区 [p.86]
- 单位增益点 (*Unity Gain Points*)：斜率 = 1 的位置，定义了 V_{IL} 和 V_{IH} [p.86–87]

V_{DD} 缩放 ($V_{DD} \text{ Scaling}$) [p.88–89]

- 1970–1980 年代: $V_{DD} = 5\text{V}$ [p.88]
- 现代芯片持续降低 V_{DD} : 3.3V, 2.5V, 1.8V, 1.5V, 1.2V, 1.0V ... [p.88]
- 目的: 避免烧毁微小晶体管 (avoid frying tiny transistors) + 节省功耗 (save power) [p.88]
- 注意: 连接不同供电电压的芯片时要小心 [p.88]

逻辑系列示例 [p.90]

常见的逻辑系列包括 TTL (Transistor-Transistor Logic)、CMOS (Complementary MOS) 等, 各系列有不同的电压参数和速度特性。

CMOS 晶体管 [p.91–113]

晶体管基础 ($Transistors$) [p.91–92]

- 逻辑门由晶体管构成 [p.91]
- 3 端口电压控制开关 (3-ported voltage-controlled switch): g (gate, 栅极), d (drain, 漏极), s (source, 源极) [p.91]
- 当 $g = 1$ 时, d 和 s 连通 (ON); $g = 0$ 时断开 (OFF) [p.91]
- **Robert Noyce (1927–1990)**: 仙童半导体 (Fairchild) 和英特尔 (Intel) 的联合创始人, 集成电路 (integrated circuit) 的共同发明人 [p.92]

硅半导体 ($Silicon$) [p.93]

- 纯硅 (pure silicon) 是差导体 (无自由电荷) [p.93]
- 掺杂硅 (doped silicon) 是良导体 (有自由电荷) [p.93]
- **n 型 (n -type)**: 自由负电荷 (电子, electrons) — 掺入砷 (As) 等 [p.93]
- **p 型 (p -type)**: 自由正电荷 (空穴, holes) — 掺入硼 (B) 等 [p.93]

MOS 晶体管结构 [p.94]

- MOS = Metal Oxide Silicon (金属氧化物硅)
- 结构: 多晶硅栅极 (Polysilicon gate) + 二氧化硅绝缘层 (SiO_2 insulator) + 掺杂硅 (doped silicon) [p.94]
- 三个端口: source(源极), gate(栅极), drain(漏极) [p.94]

nMOS 晶体管 [p.95]

- Gate = 0: OFF, 源漏之间无连接 [p.95]
- Gate = 1: ON, 源漏之间形成导电沟道 (channel) [p.95]
- nMOS 传递 0 好 (good 0), 传递 1 差, 因此源极接 GND [p.98]

pMOS 晶体管 [p.96–97]

- 与 nMOS 相反: Gate = 0 时 ON, Gate = 1 时 OFF [p.96]
- pMOS 是在 n 型衬底上的 p 沟道, 靠空穴 (holes) 流动运送电流 [p.96]
- pMOS 传递 1 好 (good 1), 传递 0 差, 因此源极接 V_{DD} [p.98]

CMOS 门结构 [p.98, 106]

- CMOS = Complementary MOS (互补 MOS), 结合 nMOS 和 pMOS
- 标准结构:
 - 上拉网络 (*pull-up network*): pMOS, 连接输出到 V_{DD} (输出 1) [p.98]
 - 下拉网络 (*pull-down network*): nMOS, 连接输出到 GND (输出 0) [p.98]
- 上下拉网络互补: 同一时刻只有一个网络导通, 避免短路 [p.106]

CMOS NOT 门 [p.99–100]

- 结构: 1 个 pMOS (P1) + 1 个 nMOS (N1) [p.99]
- $A = 0$: P1 ON, N1 OFF $\Rightarrow Y = 1$ (V_{DD}) [p.100]
- $A = 1$: P1 OFF, N1 ON $\Rightarrow Y = 0$ (GND) [p.100]
- 完美实现 $Y = \bar{A}$ [p.100]

CMOS NAND 门 [p.102–103]

- 结构：2 个并行 pMOS (P1, P2) + 2 个串联 nMOS (N1, N2) [p.102]
- 下拉网络 (nMOS 串联)：两个 nMOS 都 ON ($A = 1, B = 1$) 时才拉低 [p.103]
- 上拉网络 (pMOS 并联)：任一 pMOS ON 就拉高 [p.103]
- 真值表验证： $Y = \overline{AB}$ [p.103]
- 设计规律：上拉网络与下拉网络对偶 (dual)——nMOS 串联对应 pMOS 并联，nMOS 并联对应 pMOS 串联

NOR 门 [p.107–108]

- 与 NAND 对偶：nMOS 并联 (任一下拉即输出 0)，pMOS 串联 (两个都 OFF 才输出 1) [p.108]
- NOR3：三个并联 nMOS + 三个串联 pMOS [p.107–108]

AND 门实现 [p.109–110]

- AND 门 = NAND 门 + NOT 门 [p.110]
- 即 CMOS NAND 后接一个 CMOS NOT [p.110]

传输门 (*Transmission Gates*) [p.111]

- nMOS 传递 1 差，pMOS 传递 0 差 [p.111]
- 传输门：并联 nMOS 和 pMOS，同时提供两种载流子通路 [p.111]
- EN=1 时，开关导通，A 和 B 连通 (0 和 1 都传得好) [p.111]
- EN=0 时，开关断开，A 与 B 不连通 [p.111]

伪 nMOS 门 (*Pseudo-nMOS Gates*) [p.112–113]

- 用弱 pMOS 晶体管 (始终导通, weak pull-up) 替代上拉网络 [p.112]
- pMOS 只在 nMOS 下拉网络不拉低时才将输出拉高 [p.112]
- 例：Pseudo-nMOS NOR4 [p.113]
- 优点：晶体管数量少；缺点：nMOS 下拉网络导通时存在静态功耗 [p.112]

摩尔定律 [p.114–115]

Gordon Moore [p.114]

- 1968 年与 Robert Noyce 共同创立 Intel [p.114]
- 摩尔定律 (*Moore's Law*): 1965 年观察——计算机芯片上的晶体管数量每年翻倍 [p.114]
- 1975 年修正为每两年翻倍 [p.114]

摩尔定律的影响 [p.115]

- 推动半导体行业呈指数增长
- Cringely 的比喻: 如果汽车工业遵循同样的发展周期, 劳斯莱斯今天将花费 \$100, 每加仑行驶 100 万英里, 但每年爆炸一次 [p.115]

功耗分析 [p.116–119]

概述 [p.116]

- 功耗 (Power) = 单位时间消耗的能量 [p.116]
- 两大类: 动态功耗 (*Dynamic Power*) 和 静态功耗 (*Static Power*) [p.116]

动态功耗 (*Dynamic Power Consumption*) [p.117]

- 来源: 对晶体管栅极电容 (gate capacitances) 充电 [p.117]
- 对电容 C 充电到 V_{DD} 需要能量 CV_{DD}^2 [p.117]
- 电路以频率 f 运行时, 晶体管每秒切换 f 次 [p.117]
- 电容每秒充电 $f/2$ 次 (从 1 放电到 0 是免费的, 不耗能) [p.117]
- 动态功耗公式:

$$P_{\text{dynamic}} = \frac{1}{2}CV_{DD}^2f$$

[p.117]

静态功耗 (*Static Power Consumption*) [p.118]

- 来源：晶体管不切换时仍然消耗功率 [p.118]
- 由静态供电电流 I_{DD} （也叫泄漏电流 (*leakage current*)）引起 [p.118]
- 静态功耗公式：

$$P_{\text{static}} = I_{DD} \cdot V_{DD}$$

[p.118]

总功耗

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{static}} = \frac{1}{2} C V_{DD}^2 f + I_{DD} V_{DD}$$

功耗计算示例 [p.119]

- 无线手持电脑，参数： $V_{DD} = 1.2\text{V}$, $C = 20\text{nF}$, $f = 1\text{GHz}$, $I_{DD} = 20\text{mA}$ [p.119]
- 动态功耗： $P_{\text{dynamic}} = \frac{1}{2} \times 20\text{nF} \times (1.2\text{V})^2 \times 1\text{GHz} = 14.4\text{W}$ [p.119]
- 静态功耗： $P_{\text{static}} = 20\text{mA} \times 1.2\text{V} = 0.024\text{W}$ [p.119]
- 总功耗 $\approx 14.4\text{W}$ （动态功耗占绝对主导） [p.119]

习题示例 [p.120]

全加器设计 (*Full Adder Design*)

- 题目：依据三 Y 原则，用 nMOS 和 pMOS 设计带进位的 1 位全加器，并用 Digital 工具仿真 [p.120]
- 输入： A, B, C_{in} （被加数、加数、进位输入），输出： S, C_{out} （和、进位输出） [p.120]

真值表与逻辑函数 [p.120]:

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

逻辑表达式 [p.120]:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

$$A \oplus B = A\overline{B} + \overline{A}B$$

设计提示:

- 使用层次化: 先设计 XOR 门模块
- 使用模块化: XOR 模块接口清晰
- 使用规整化: 重复使用 XOR 模块

— 第 1 章完 —

本笔记覆盖 DDCA ARM Edition 第 1 章全部 120 页, 包含数字设计基础的全部核心考点。